

```
import pandas as pd
```

```
df = pd.read_csv("/content/stock_prices.csv")
df.head()
```

	date	ticker	open	high	low	close	volume	returns_pct
0	2022-01-03	RELIANCE	2476.84	2496.70	2422.91	2459.80	4940428	-1.6079
1	2022-01-04	RELIANCE	2443.34	2497.22	2423.41	2460.31	6310409	0.0207
2	2022-01-05	RELIANCE	2467.38	2481.76	2408.41	2445.09	2920856	-0.6189
3	2022-01-06	RELIANCE	2450.52	2472.16	2399.09	2435.63	2458621	-0.3868
4	2022-01-07	RELIANCE	2415.56	2439.90	2367.79	2403.85	4011166	-1.3049

Next steps:

[Generate code with df](#)
[New interactive sheet](#)

```
stock_df = df[df['ticker'] == 'RELIANCE'].copy()
stock_df.sort_values('date', inplace=True)
```

```
stock_df['date'] = pd.to_datetime(stock_df['date'])
stock_df.set_index('date', inplace=True)
```

```
data = stock_df[['close']]
```

```
import numpy as np
```

```
WINDOW_SIZE = 30
```

```
X, y = [], []
```

```
for i in range(len(data) - WINDOW_SIZE):
    X.append(data.iloc[i:i+WINDOW_SIZE].values)
    y.append(data.iloc[i+WINDOW_SIZE].values)
```

```
X = np.array(X)
```

```
y = np.array(y)
```

```
train_size = int(len(X) * 0.7)
```

```
val_size = int(len(X) * 0.15)
```

```
X_train = X[:train_size]
```

```
y_train = y[:train_size]

X_val = X[train_size:train_size+val_size]
y_val = y[train_size:train_size+val_size]

X_test = X[train_size+val_size:]
y_test = y[train_size+val_size:]
```

Sub-step 2 — Chat Logs

```
chat_df = pd.read_csv("/content/chat_logs.csv")
chat_df.head()
```

	chat_id	timestamp	duration_min	num_turns	customer_sentiment	priority
0	CHT000000	25/05/2024 08:00	11	15	angry	canceled
1	CHT000001	2024-01-18 00:00:00	24	11	frustrated	replied
2	CHT000002	30/01/2024 19:00	26	17	frustrated	replied
3	CHT000003	2024-11-02 20:00:00	15	8	angry	replied
4	CHT000004	2024-12-28 03:00:00	9	17	neutral	technical

Next steps:

[Generate code with chat_df](#)[New interactive sheet](#)

```
chat_df['timestamp'] = pd.to_datetime(chat_df['timestamp'], errors='coerce')
```

```
# Drop invalid timestamps
chat_df = chat_df.dropna(subset=['timestamp'])
```

```
/tmp/ipykernel_21067/896453431.py:1: UserWarning: Parsing dates in %d/%m/%Y %
chat_df['timestamp'] = pd.to_datetime(chat_df['timestamp'], errors='coerce')
```

```
chat_df.info()
chat_df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
Index: 583 entries, 0 to 2489
```

```
Data columns (total 10 columns):
```

#	Column	Non-Null Count	Dtype
0	chat_id	583 non-null	object
1	timestamp	583 non-null	datetime64[ns]
2	duration_min	583 non-null	int64
3	num_turns	583 non-null	int64
4	customer_sentiment	566 non-null	object
5	primary_intent	583 non-null	object
6	resolution_status	568 non-null	object
7	tenure_months	583 non-null	int64
8	product_tier	583 non-null	object
9	churned_30d	583 non-null	int64

```
dtypes: datetime64[ns](1), int64(4), object(5)
```

```
memory usage: 50.1+ KB
```

	timestamp	duration_min	num_turns	tenure_months	churned_30d
count	583	583.000000	583.000000	583.000000	583.000000
mean	2024-06-27 00:44:21.406518016	17.691252	10.530017	17.919383	0.190395
min	2024-01-02 03:00:00	1.000000	1.000000	1.000000	0.000000
25%	2024-03-24 15:00:00	7.000000	6.000000	5.000000	0.000000
50%	2024-06-28 01:00:00	12.000000	11.000000	13.000000	0.000000
75%	2024-09-23 15:30:00	21.000000	15.000000	25.000000	0.000000
max	2024-12-29 18:00:00	120.000000	19.000000	84.000000	1.000000
std	NaN	18.405439	5.412412	17.065228	0.392949

```
chat_df.info()
```

```
chat_df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
Index: 583 entries, 0 to 2489
```

```
Data columns (total 10 columns):
```

#	Column	Non-Null Count	Dtype
0	chat_id	583 non-null	object
1	timestamp	583 non-null	datetime64[ns]
2	duration_min	583 non-null	int64
3	num_turns	583 non-null	int64
4	customer_sentiment	566 non-null	object
5	primary_intent	583 non-null	object
6	resolution_status	568 non-null	object
7	tenure_months	583 non-null	int64
8	product_tier	583 non-null	object
9	churned_30d	583 non-null	int64

```
dtypes: datetime64[ns](1), int64(4), object(5)
```

```
memory usage: 50.1+ KB
```

	timestamp	duration_min	num_turns	tenure_months	churned_30d
count	583	583.000000	583.000000	583.000000	583.000000
mean	2024-06-27 00:44:21.406518016	17.691252	10.530017	17.919383	0.190395
min	2024-01-02 03:00:00	1.000000	1.000000	1.000000	0.000000
25%	2024-03-24 15:00:00	7.000000	6.000000	5.000000	0.000000
50%	2024-06-28 01:00:00	12.000000	11.000000	13.000000	0.000000
75%	2024-09-23 15:30:00	21.000000	15.000000	25.000000	0.000000
max	2024-12-29 18:00:00	120.000000	19.000000	84.000000	1.000000
std	NaN	18.405439	5.412412	17.065228	0.392949

```
duration = chat_df.groupby('chat_id')['timestamp'].agg(['min', 'max'])
duration['duration'] = (duration['max'] - duration['min']).dt.total_seconds()
```

```
chat_df = chat_df.sort_values(['chat_id', 'timestamp'])
chat_df['response_time'] = chat_df.groupby('chat_id')['timestamp'].diff().dt
```

MediumSub-step 3 — LSTM for Stock Prediction

```
import torch
import torch.nn as nn

class LSTMModel(nn.Module):
    def __init__(self, input_size=1, hidden_size=64, num_layers=2):
```

```
super(LSTMModel, self).__init__()

self.lstm = nn.LSTM(input_size, hidden_size, num_layers, batch_first)
self.fc = nn.Linear(hidden_size, 1)

def forward(self, x):
    out, _ = self.lstm(x)
    out = out[:, -1, :] # last timestep
    out = self.fc(out)
    return out
```

```
model = LSTMModel()
criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

```
EPOCHS = 10

for epoch in range(EPOCHS):
    model.train()

    inputs = torch.tensor(X_train, dtype=torch.float32)
    targets = torch.tensor(y_train, dtype=torch.float32)

    outputs = model(inputs)
    loss = criterion(outputs, targets)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    print(f"Epoch {epoch+1}, Loss: {loss.item()}")
```

```
Epoch 1, Loss: 5817160.0
Epoch 2, Loss: 5816848.5
Epoch 3, Loss: 5816542.0
Epoch 4, Loss: 5816229.0
Epoch 5, Loss: 5815912.0
Epoch 6, Loss: 5815585.0
Epoch 7, Loss: 5815243.0
Epoch 8, Loss: 5814885.0
Epoch 9, Loss: 5814503.0
Epoch 10, Loss: 5814093.5
```

```
model.eval()

with torch.no_grad():
    test_inputs = torch.tensor(X_test, dtype=torch.float32)
    predictions = model(test_inputs).numpy()
```

```
from sklearn.metrics import mean_squared_error
import numpy as np
```

```
rmse = np.sqrt(mean_squared_error(y_test, predictions))
print("RMSE:", rmse)
```

RMSE: 1796.8627256202114

Sub-step 4 — Churn Prediction

```
# Select relevant features for churn predic
features = ['duration_min', 'num_turns', 'c
target = 'churned_30d'
```

```
df_churn = chat_df[features + [target]].cop
```

```
# Handle missing values in categorical feat
df_churn['customer_sentiment'].fillna('unkn
df_churn['resolution_status'].fillna('unkno
```

```
# One-hot encode categorical features
df_churn = pd.get_dummies(df_churn, columns
```

```
# Define X and y for the tabular model
X_tab = df_churn.drop(columns=[target])
y_tab = df_churn[target]
```

```
display(X_tab.head())
```

/tmp/ipykernel_21067/1557215353.py:8: FutureWarning: A value is trying to be
The behavior will change in pandas 3.0. This inplace method will never work b
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.

```
df_churn['customer_sentiment'].fillna('unknown', inplace=True)
/tmp/ipykernel_21067/1557215353.py:9: FutureWarning: A value is trying to be
The behavior will change in pandas 3.0. This inplace method will never work b
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.
```

```
df_churn['resolution_status'].fillna('unknown', inplace=True)
```

	duration_min	num_turns	tenure_months	customer_sentiment_confused	cust
0	11	15	3		False
2	26	17	42		False
6	8	10	20		False
7	18	5	14		False
14	6	11	12		False

```
from sklearn.model_selection import train_test_split
```

```
# Split data into training and test sets
X_train_tab, X_test_tab, y_train_tab, y_test_tab = train_test_split(X_tab, y

# Further split training data into training and validation sets
X_train_tab, X_val_tab, y_train_tab, y_val_tab = train_test_split(X_train_ta

# Extract customer_ids for the test set from the original chat_df using the
customer_ids_test = chat_df.loc[X_test_tab.index, 'chat_id'].values
```

```
from sklearn.linear_model import LogisticRegression
```

```
model_tab = LogisticRegression()
model_tab.fit(X_train_tab, y_train_tab)
```

```
preds_tab = model_tab.predict(X_test_tab)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regres

```
n_iter_i = _check_optimize_result(
```

```
from sklearn.metrics import f1_score
```

```
f1 = f1_score(y_test_tab, preds_tab)
print("F1 Score:", f1)
```

```
F1 Score: 0.375
```

Sub-step 5 — Business Output

```
probs = model_tab.predict_proba(X_test_tab)[: ,1]
```

```
import pandas as pd
```

```
results = pd.DataFrame({
    "customer_id": customer_ids_test,
    "churn_probability": probs
})
```

```
results = results.sort_values(by="churn_probability", ascending=False)
display(results.head())
```

	customer_id	churn_probability	
34	CHT001841	0.776058	
113	CHT001889	0.762984	
88	CHT001567	0.749431	
46	CHT001310	0.734043	
14	CHT001992	0.732258	

```
threshold = 0.3 # example
```

```
selected = results[results['churn_probability'] > threshold]
display(selected.head())
```

	customer_id	churn_probability	
34	CHT001841	0.776058	
113	CHT001889	0.762984	
88	CHT001567	0.749431	
46	CHT001310	0.734043	
14	CHT001992	0.732258	

hard -Sub-step 6 — Autoregressive Baseline vs LSTM

```
WINDOW_SIZE = 30
```

```
def autoregressive_predict(X):
    return X.mean(axis=1)
```

```
baseline_preds = autoregressive_predict(X_test)
```

```
from sklearn.metrics import mean_squared_error
import numpy as np
```

```
baseline_rmse = np.sqrt(mean_squared_error(y_test, baseline_preds))
print("Baseline RMSE:", baseline_rmse)
```

```
Baseline RMSE: 76.45704862797214
```

```
print("LSTM RMSE:", rmse)
print("Baseline RMSE:", baseline_rmse)
```

```
LSTM RMSE: 1796.8627256202114
Baseline RMSE: 76.45704862797214
```


Sub-step 7 — Manual BPTT + Vanishing Gradient

```

import numpy as np

np.random.seed(0)

T = 10 # sequence length
input_size = 1
hidden_size = 2

Wx = np.random.randn(hidden_size, input_size)
Wh = np.random.randn(hidden_size, hidden_size)

h = np.zeros((T, hidden_size))
x = np.random.randn(T, input_size)

for t in range(T):
    prev_h = h[t-1] if t > 0 else np.zeros(hidden_size)
    h[t] = np.tanh(Wx @ x[t] + Wh @ prev_h)

```

```

dWh = np.zeros_like(Wh)
dWx = np.zeros_like(Wx)

dh_next = np.zeros(hidden_size)

for t in reversed(range(T)):
    dh = (1 - h[t]**2) + dh_next # tanh d

    prev_h = h[t-1] if t > 0 else np.zeros(

    dWh += np.outer(dh, prev_h)
    dWx += np.outer(dh, x[t])

    dh_next = Wh.T @ dh

```

```

lengths = [5, 10, 20, 30, 50]
grad_norms = []

```

```

for T in lengths:

```